

Introduction to complexity theory

What is this for?

- In this course you've seen lots of problems.
- Some had solutions that seemed pretty good (e.g. shortest path, minimum spanning tree, longest common subsequence,...)
- Some didn't seem so good (e.g. travelling salesman problem)
- Is that just because Steve isn't smart enough?
- Can we give some kind of justification that we can't do better?
- And btw what do we mean by good/not so good?

Decisions, decisions

- Lots of the problems we have talked about have been *optimisation* problems: “find the smallest (or largest, or cheapest) [X] such that [Y]”.
- We want one uniform type of problem: *decision* problems, where the answer is yes or no.

Example: TSP

TSP (optimisation version)

Input: A complete directed weighted graph G .

Output: A Hamiltonian cycle of G of minimum weight.

TSP (decision version)

Input: A complete directed weighted graph G **and an integer T** .

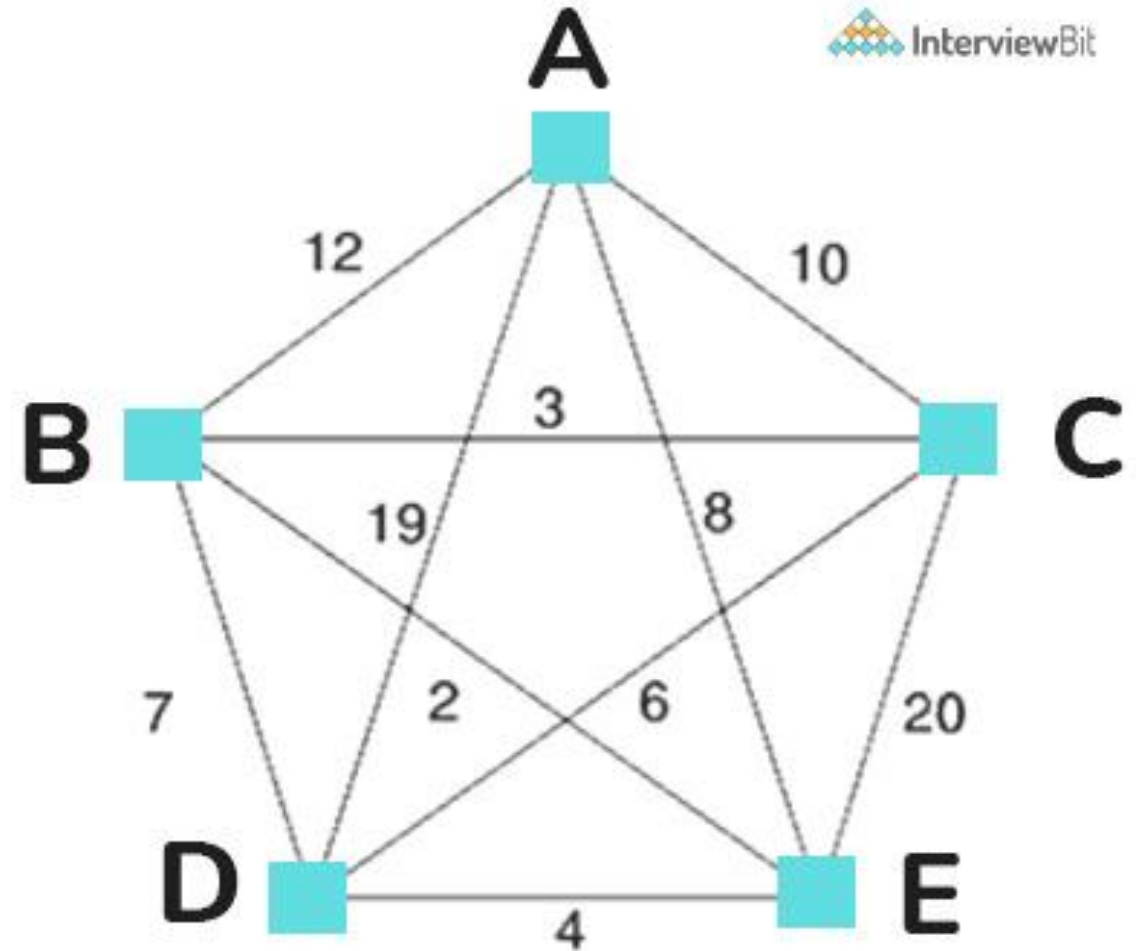
Output: Yes if G has a Hamiltonian cycle of weight at most T , No otherwise.

Why are these kind of equivalent?

Example

Consider the following weighted graph

Is there a Hamiltonian cycle of weight less than 50?



Example: MST

To do now: write down the optimisation version of the minimum spanning tree problem. Write down the decision version.

Exercises

- CLRS 34.1.1,34.1.2
- Pick some of the problems from the lectures on greedy algorithms and DP. Write down optimisation and decision versions of them.

Languages

- Another viewpoint: a decision problem is just a subset of the possible inputs – the subset for which the answer is “yes”
- For example, what subset does MST correspond to?
- If we think of our input as being written down as a string (e.g. write down the adjacency list of a graph, etc.) then the problem is just a set of strings (that describe the “yes” instances).
- We call a set of strings a *language*.

Tractable problems: the
class P

Tractability

- What should it mean to say a problem has an efficient solution (it is “tractable”)?
- When we look at asymptotic runtimes, we “zoom out” and say we don’t care about constant factors, so $5n^2$ is the same thing as $1000000n^2$.
- Now we’re going to zoom out even further and say we don’t care about polynomial factors.
- So n, n^2, n^{10}, n^{1000} are all good.
- What is n ? The **size** of the input (NOT the numbers in the input)

P

Definition: a language L is *in* P if there exists an algorithm A and a polynomial p such that:

- A decides membership in L : on input l A outputs yes if $l \in L$ and no otherwise.
- A runs in polynomial time: on input l A runs in time at most $p(|l|)$.
- A is deterministic: randomness is not allowed.

(We can think of P as a set of languages and just write $L \in P$)

P

How do I show that a problem is in P?

Option 1: Just do it! Write down the algorithm A.

Option 2: *Reduce* the problem to one I already know is in P.

Reductions

Definition: a polynomial-time (Karp) reduction from a language(/problem) L_1 to a language(/problem) L_2 is a polynomial-time computable function f mapping instances of L_1 to instances of L_2 , such that $l \in L_1 \Leftrightarrow f(l) \in L_2$. We write $L_1 \rightarrow L_2$.

Observation: if there is a polynomial-time reduction $L_1 \rightarrow L_2$ and $L_2 \in P$, then $L_1 \in P$.

Important: what I have to show is exactly how to map an instance of problem 1 to an instance of problem 2, not just explain why the two problems are similar!

Reductions

Example: Shortest common supersequence

Input: Two strings s_1, s_2 .

Output: The length of the shortest string t such that s_1, s_2 are subsequences of t .

Formulate the decision version of this problem. Show that it is in P by finding a reduction to a problem you know is in P.

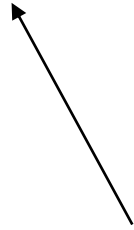
Intractible problems

- OK, that's how we show a problem **is** in P.
- How about problems where we **can't** find an efficient algorithm?
How do we show they're **not** in P?
- That seems a bit harder...

P vs NP

The million-dollar question

- “Can problems that can be efficiently checked be efficiently solved?”



P

???



NP

- Just like we defined the class P to formalise the idea of problems that can be efficiently solved, we will define the class NP to formalise the idea of problems whose solutions can be efficiently checked.
- We will think about two possible definitions:
 - Give an answer which we can check.
 - Give ourselves psychic powers.
- ...but we will see they are equivalent!

NP 1

- A problem/language is in NP if membership has efficiently-verifiable *certificates*.
- Formally, a *verification* algorithm B is an algorithm which takes as input a pair of strings (x,y) , where x is a problem instance and y is a 'certificate'
- B is *sound* if whenever $x \notin L$ there is no certificate which will satisfy B , i.e. B will output 0 on input (x,y) for any y
- B is *complete* if whenever $x \in L$ there is a certificate which will convince B , i.e. there exists some y (of size bounded by a polynomial $q(n)$) such that B will output 1 on input (x,y) (**but y may be hard to find**)
- A language L is in NP if it has a verifier which is (1) deterministic polynomial-time (2) sound and (3) complete
- Note that this treats "yes" and "no" differently

NP 1

- For example: why is the TSP problem in NP?
- The certificate is a description of a Hamiltonian cycle of weight at most w
- Another example: factoring numbers.
- Pre-problem: how do we make this a decision problem?
- Answer: an instance is a pair (n,k) , where the question is “does n have a factor of size at most k ?” (how could you use this to make an algorithm to actually compute the factors?)
- So now a certificate is a factor $l \leq k$
- So the verifier B accepts $((n,k),l)$ if (1) $l \leq k$ and (2) n is divisible by l

NP 1

- Note any problem in P satisfies this definition, i.e. $P \subseteq NP$
- The language is in P : there is a deterministic polynomial-time algorithm A that returns 1 if $x \in L$ and 0 if not
- Now we need to think of a sound, complete polynomial-time verification algorithm B
- Solution: given (x,y) ignore the certificate y , run A on x and output the answer

NP 2

- Second option: give ourselves psychic powers
- We say that an algorithm is non-deterministic if it uses the following operation:
 - $\text{choose}(b)$: chooses a bit b arbitrarily/'at random'/**non-deterministically**
- Can be used as many times as desired, so can choose nondeterministically between an arbitrary number of options
- We will now think of our algorithm as returning either “yes” or “don't know”
- A language L is in NP if there is a **Non-deterministic Polynomial-time** algorithm C such that:
 - If $x \in L$ then C can return 1 on input x , for some possible outcomes of the choose calls
 - If $x \notin L$ then C will always return 0 on input x

NP 2

- Why does TSP satisfy NP 2?
- At each step nondeterministically pick a vertex we haven't visited yet.
- If in the end the total cost is $\leq w$ return 1, otherwise 0.
- Why does this satisfy the conditions?

Equivalence

- Why are these equivalent definitions?
- We have two things to prove:
 - if L is in NP1 then it is in NP2
 - if L is in NP2 then it is in NP1

If L is in NP1 then it is in NP2

- We have a verifier B for certificates and we have to make a non-deterministic algorithm C
- Solution: run $\text{choose}(b)$ repeatedly $q(n)$ times to generate a certificate y , then run B on (x,y) and return the answer
- Two things to check:
 - If $x \in L$ then there is a certificate y that B will accept, so if the choose calls generated this y then C will return 1
 - If $x \notin L$ then B will always return 0 whatever y is, hence C will always return 0

If L is in NP2 then it is in NP1

- We have a non-deterministic algorithm C and we have to make a verifier B
- Solution: given a pair (x,y) , run C and when $\text{choose}(b)$ is called for the i th time, return the i th bit of y .
- Two things to check:
 - If $x \in L$ then there is a sequence of values for choose such that C will return 1, so when these are written as the certificate y , B will return 1 on (x,y)
 - If $x \notin L$ then C will always return 0 hence so will B

coNP

- Note that NP has a big asymmetry between 'yes' and 'no' answers
- NP1: the verifier has to verify 'yes' answers
- NP2: the nondeterministic machine says 'yes' or 'don't know'
- We could think of doing things the other way round
- coNP: the class of languages L such that L^c (L-complement) is in NP

Exercises

Formulate each of the following as a decision problem, and show that it is in NP:

- Shortest path (given a graph G and vertices u and v of G , what is the length of the shortest path from u to v in G)
- Knapsack (given a collection of objects each with a weight w_i and a value v_i , and a total carrying capacity C , what is the maximum total value I can put in my knapsack)
- Chromatic number (given a graph G , what is the fewest number of colours I need to colour all the vertices such that no neighbouring vertices have the same colour)

Is $P=NP$?

- We have absolutely no idea
- Well actually, almost everyone thinks not but we have no idea how to prove it
- To answer “yes” we’d need to think of an algorithm.
- To answer “no” we’d need to find some clever proof.
- Unfortunately we have no idea how to do either, and we have made ~no progress in the last 50 years 😞

Consequences

- If $P \neq NP$, then nothing really changes
- If $P = NP$ then every problem that can be written down would be easy to solve
 - Every maths proof
 - Every chemical formula
 - Every protein folding
 - Every scheduling problem
 - **Every cryptographic scheme**
- It would be world-changing (for good and ill)
- It is simply the most important open problem out there

Consequences

- The fact we have no idea how to answer this question is kind of embarrassing
- It means we have no chance of proving that any problem (in NP) is intractable, since even one would prove that $P \neq NP$ 😞
- Fortunately, we can do something almost as good
- Remarkably, we can often show that a problem is *as hard as any problem in NP* – we call this being ‘NP-hard’
- Since we strongly believe $P \neq NP$, in practice this means the problem is intractable

Hardness reductions

Reductions again

- Before we used reductions ‘for good’, to show problems were easy.
- (If I can reduce L1 to L2, and I can solve L2, then I can solve L1).
- Now we’re going to use them ‘for evil’, to show problems are hard.
- If I can reduce L1 to L2, and I believe I **can’t** solve L1, then I have to believe I **can’t** solve L2.

You might want to take a long hot bath and think about that for a bit...

NP-hard

Definition: A language L is *NP-hard* if for every language $M \in NP$ there is a polynomial time reduction $M \rightarrow L$.

(If also $L \in NP$ then we say L is *NP-complete*)

This seems like it should be ridiculously difficult to show – I need a reduction for *every* problem in NP!

But actually we can often do it – we'll see how next lecture.

Exercises

- R-17.1, R-17.7, R-17.12, R-17.14, R-17.15, C-17.4
- CLRS 34.2-3, 34.2-4, 34.2-6, 34.2-9, 34.2-10.